

Tarjan 与图连通性

Ehundategu

Giouh 大队

2026 年 7 月

Contents

连通性与 Low-Link

强连通分量

割点、桥与边双

点双连通分量

Contents

连通性与 Low-Link
连通性的层次
DFS 树与返祖边

强连通分量

割点、桥与边双

点双连通分量

Lead in

- ▶ 在树中，任意两点之间只有一条简单路径，所以删除一条边一定会破坏连通性。但是一般图中存在环，一条边或一个点失效后，原来的路径可能被另一条路径替代。

Lead in

- ▶ 在树中，任意两点之间只有一条简单路径，所以删除一条边一定会破坏连通性。但是一般图中存在环，一条边或一个点失效后，原来的路径可能被另一条路径替代。
- ▶ 因此，图连通性的核心并不是“当前能否到达”，而是“某一条路径被切断后，是否仍然能够绕行”。Tarjan 算法正是利用一次 DFS，记录每棵子树能够绕回的位置。

Definition

- ▶ 在正式讨论 Tarjan 之前，我们先约定图中“连通”的含义。

Definition

- ▶ 在正式讨论 Tarjan 之前，我们先约定图中“连通”的含义。
- ▶ 对于无向图中的两个结点 u, v ，若能够从 u 出发沿边走到 v ，则称 u, v 连通。将所有互相连通，并且不能再加入其他结点的结点放在一起，就得到了一个连通分量。

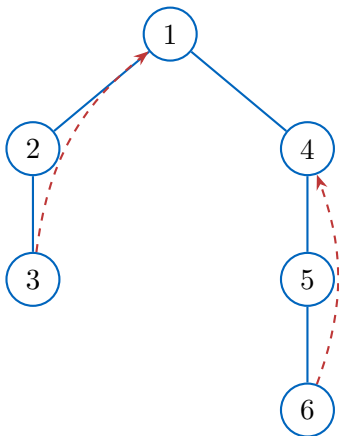
Definition

- ▶ 在正式讨论 Tarjan 之前，我们先约定图中“连通”的含义。
- ▶ 对于无向图中的两个结点 u, v ，若能够从 u 出发沿边走到 v ，则称 u, v 连通。将所有互相连通，并且不能再加入其他结点的结点放在一起，就得到了一个连通分量。
- ▶ 对于有向图，能够从 u 到达 v 并不意味着能够从 v 回到 u 。只有二者能够互相到达时，才称 u, v 强连通。由此得到的极大结点集合称为强连通分量。

Definition

- ▶ 若删除一条边会增加连通分量数，这条边称为桥；若删除一个点及其相邻边会增加连通分量数，这个点称为割点。
- ▶ 不存在桥的极大连通部分称为边双连通分量；任意两点之间至少存在两条内部点不相交路径的极大部分称为点双连通分量。
- ▶ 边双研究“能否绕开一条边”，点双研究“能否绕开一个点”，二者的判定条件只差一个等号。

Definition



- ▶ 第一次走到一个新结点的边构成 DFS 树，称为树边。
- ▶ 指向已经访问过的祖先的边称为返祖边。图中真正提供“绕行”的正是这些边。
- ▶ 后续只需要回答：一棵 DFS 子树经过树内路径与至多一条返祖边，最早能够回到哪个祖先？

Definition

- ▶ 令 dfn_u 表示结点 u 第一次被访问的次序。祖先一定比后代更早被访问，因此它的 dfn 更小。
- ▶ 在无向图中，令 low_u 表示从 u 的 DFS 子树出发，不经过 u 的父边，能够到达的最小 dfn 。

$$\text{low}_u = \min \left(\text{dfn}_u, \min_{(u,v) \text{ 为树边}} \text{low}_v, \min_{(u,v) \text{ 为返祖边}} \text{dfn}_v \right).$$

Theory

- ▶ 访问 u 时，我们只能确定 dfn_u ；只有完整搜索完儿子 v 的子树后，才知道这棵子树能够绕回哪里。
- ▶ 所以树边 (u, v) 的贡献是 low_v ，返祖边 (u, v) 的贡献则是已经确定的 dfn_v 。Tarjan 的所有判定都发生在对子树完成回溯的时刻。
- ▶ 整个过程中每个结点与每条边只被处理常数次，时间复杂度为 $\mathcal{O}(n + m)$ 。

Theory

- ▶ 有向边不能反向行走，所以“父边”不再需要排除；但是指向已经结束的强连通分量的边，也不能帮助当前结点回到栈内的祖先。
- ▶ 因此，有向图中只有指向尚未确定 SCC 的结点的边可以更新 low。这要求我们额外维护一个栈，记录当前仍可能属于同一 SCC 的结点。
- ▶ 无向图与有向图的实现细节不同，但共同点仍然是：用 low 判断当前 DFS 子树能否回到当前结点之前。

- ▶ DFS 搜索到一条指向已访问结点的边时，这个结点可能仍在当前搜索链上，也可能已经属于一个确定的 SCC。
- ▶ 只有前一种边能够把当前结点与更早的结点合并。因此，将尚未确定 SCC 的结点压入栈中，并用 $SCC[To]==0$ 判断它是否仍在栈内。
- ▶ 一个结点离栈后所属 SCC 已经确定，此后指向它的边不再参与 low 更新。

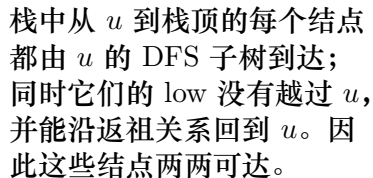
Method

对结点 u 枚举有向边 (u, v) :

- ▶ 若 v 尚未访问，先递归搜索 v ，再令 $\text{low}_u \leftarrow \min(\text{low}_u, \text{low}_v)$ 。
- ▶ 若 v 已经访问但尚未确定 SCC，说明它仍在栈中，令 $\text{low}_u \leftarrow \min(\text{low}_u, \text{dfn}_v)$ 。
- ▶ 若 v 已经离栈，则这条边不会让 u 回到当前未封闭部分，不进行更新。

Theory

- ▶ 若 $\text{low}_u < \text{dfn}_u$ ，说明 u 或它的后代能够回到更早的栈内结点，因此当前部分仍没有封闭。
- ▶ 若 $\text{low}_u = \text{dfn}_u$ ，说明 u 的 DFS 子树无法回到 u 之前。此时 u 是一个 SCC 在 DFS 树中的最高结点。
- ▶ 从栈顶不断弹出结点，直到弹出 u ，这些结点恰好构成一个 SCC。



同一次弹栈

Theory

- ▶ 弹栈时，若栈顶结点 x 与 u 不强连通，那么只能是 x 无法回到 u 。但是 x 尚未在更深处被弹出，说明它能够沿栈内边回到不晚于 u 的结点，与假设矛盾。
- ▶ 反过来，若一个与 u 强连通的结点没有被弹出，它要么位于 u 之下而仍在栈中，要么已经提前离栈。前者会被弹出；后者意味着它无法再回到 u ，同样矛盾。
- ▶ 所以一次弹栈既不会混入其他 SCC，也不会遗漏当前 SCC 的结点。

Theory

- ▶ 将每个 SCC 缩成一个结点，并保留不同 SCC 之间的边。
- ▶ 若缩点后仍然存在有向环，那么环上的所有 SCC 可以沿环相互到达，它们的并集仍然强连通，与每个 SCC 的极大性矛盾。
- ▶ 因此缩点图一定是 DAG。许多一般有向图问题可以先缩点，再在 DAG 上进行拓扑排序或动态规划。

Example

P3387 【模板】缩点

- ▶ 给定一张有 n 个结点和 m 条边的有向图，每个结点都有一个权值。
- ▶ 可以重复经过一条边或一个结点，但同一个结点的权值只会计算一次。求一条路径，使其经过结点的权值之和最大。

Analyse

- ▶ 对每个 SCC C ，将其中所有结点的权值相加，记为 W_C 。原图中连接两个不同 SCC 的边，在缩点图中保留为一条有向边。
- ▶ SCC 内部的移动顺序不会影响这一分量的总贡献。缩点后不再存在环，所以每个分量只需要在 DAG 上处理一次。
- ▶ 原图中的一条合法路径与缩点图中的一条路径对应。原问题的答案就是缩点图上结点权值之和的最大值。

Theory

令 f_v 表示以缩点 v 结尾的路径能够取得的最大权值和。若 v 存在前驱，则

$$f_v = W_v + \max_{(u,v) \in E'} f_u.$$

若 v 没有前驱，则令 $f_v = W_v$ 。由于缩点图是 DAG，只要按照拓扑序计算，每次转移时所有前驱的答案都已经确定。

Method

- ▶ 先用 Tarjan 求出每个结点所属的 SCC，并将原结点权值累加到对应缩点上。
- ▶ 枚举原图中的边。若两个端点属于不同 SCC，就在缩点图中加入对应的有向边，并统计入度。重复边不影响正确性。
- ▶ 将所有入度为 0 的缩点加入队列。每次取出 u 后，用 $f_u + W_v$ 更新后继 v 。总时间复杂度为 $\mathcal{O}(n + m)$ 。

Code

```
1 #include <stack>
2 #include <cstdio>
3 #include <cstring>
4 #include <algorithm>
5 #define MAXN 200010
6 #define MAXM 500010
7 using namespace std;
8 int n,m,Head[MAXN],Total=0,Dfn[MAXN],Low[MAXN],SCC[MAXN];
9 int cnt=0,cnd=0,In1,In2;
10 stack <int> S;
11 struct edge{
12     int Ed,Next;
13 }Edge[MAXM];
14 void Edge_Add(int St,int Ed){
15     Edge[++Total]={Ed,Head[St]};
16     Head[St]=Total;
17 }
18 void Tarjan(int Now){
```

Code

```
1    Dfn[Now]=Low[Now]=++cnt;
2    S.push(Now);
3    for(int i=Head[Now];i;i=Edge[i].Next){
4        int To=Edge[i].Ed;
5        if(!Dfn[To]){
6            Tarjan(To);
7            Low[Now]=min(Low[Now],Low[To]);
8        }
9        else if(!SCC[To]) Low[Now]=min(Low[Now],Dfn[To]);
10   }
11   if(Low[Now]==Dfn[Now]){
12       cnd++;
13       while(S.top()!=Now){
14           SCC[S.top()]=cnd;
15           S.pop();
16       }
17       SCC[Now]=cnd;S.pop();
18   }
```

Code

```

1  }
2  int main(){
3      scanf("%d%d",&n,&m);
4      for(int i=1;i<=m;i++){
5          scanf("%d%d",&In1,&In2);
6          Edge_Add(In1,In2);
7      }
8      for(int i=1;i<=n;i++) if(!Dfn[i]) Tarjan(i);
9      printf("%d\n",cnd);
10     for(int i=1;i<=n;i++) printf("%d ",SCC[i]);
11     return 0;
12 }

```

Code

```
1 #include <stack>
2 #include <queue>
3 #include <cstdio>
4 #include <climits>
5 #include <algorithm>
6 #define MAXN 200010
7 #define MAXM 500010
8 using namespace std;
9 int n,m,Head[MAXN],Head2[MAXN],Total=0,Total2=0;
10 int Dfn[MAXN],Low[MAXN],SCC[MAXN],Deg[MAXN],cnt=0,cnd=0;
11 int In1,In2;
12 long long Value[MAXN],Sum[MAXN],DP[MAXN],Ans=LLONG_MIN;
13 stack <int> S;
14 queue <int> Q;
15 struct edge{
16     int St,Ed,Next;
17 }Edge[MAXN],Edge2[MAXN];
18 void Edge_Add(int St,int Ed){
```

Code

```

1      Edge[++Total]={St,Ed,Head[St]};
2      Head[St]=Total;
3  }
4  void Edge_Add2(int St,int Ed){
5      Edge2[++Total2]={St,Ed,Head2[St]};
6      Head2[St]=Total2;
7  }
8  void Tarjan(int Now){
9      Dfn[Now]=Low[Now]=++cnt;
10     S.push(Now);
11     for(int i=Head[Now];i;i=Edge[i].Next){
12         int To=Edge[i].Ed;
13         if(!Dfn[To]){
14             Tarjan(To);
15             Low[Now]=min(Low[Now],Low[To]);
16         }
17         else if(!SCC[To]) Low[Now]=min(Low[Now],Dfn[To]);
18     }

```

Code

```

1      if(Low[Now]==Dfn[Now]){
2          cnd++;
3          while(S.top()!=Now){
4              SCC[S.top()]=cnd;
5              S.pop();
6          }
7          SCC[Now]=cnd;S.pop();
8      }
9  }
10 int main(){
11     scanf("%d%d",&n,&m);
12     for(int i=1;i<=n;i++) scanf("%lld",&Value[i]);
13     for(int i=1;i<=m;i++){
14         scanf("%d%d",&In1,&In2);
15         Edge_Add(In1,In2);
16     }
17     for(int i=1;i<=n;i++) if(!Dfn[i]) Tarjan(i);
18     for(int i=1;i<=n;i++) Sum[SCC[i]]+=Value[i];

```

Code

```

1    for(int i=1;i<=Total;i++){
2        int St=SCC[Edge[i].St],Ed=SCC[Edge[i].Ed];
3        if(St==Ed) continue;
4        Edge_Add2(St,Ed);Deg[Ed]++;
5    }
6    for(int i=1;i<=cnd;i++){
7        DP[i]=Sum[i];
8        if(!Deg[i]) Q.push(i);
9    }
10   while(!Q.empty()){
11       int Now=Q.front();Q.pop();
12       Ans=max(Ans,DP[Now]);
13       for(int i=Head2[Now];i;i=Edge2[i].Next){

```

Code

```

1      int To=Edge2[i].Ed;
2      DP[To]=max(DP[To],DP[Now]+Sum[To]);
3      if(!--Deg[To]) Q.push(To);
4  }
5  }
6  printf("%lld\n",Ans);
7  return 0;
8  }
```


Definition

- ▶ 2-SAT 要求为若干个布尔变量选择真假，使每条只涉及两个取值的限制同时成立。
- ▶ 对每个布尔变量 x_i 建立两个结点，分别表示 $x_i = 0$ 与 $x_i = 1$ 。一条蕴含 $a \Rightarrow b$ 对应一条有向边。
- ▶ 若限制“不允许 a 与 b 同时成立”，等价于加入 $a \Rightarrow \neg b$ 与 $b \Rightarrow \neg a$ 。
- ▶ 当 $x_i = 0$ 与 $x_i = 1$ 位于同一个 SCC 时，二者互相推出，变量无论取何值都会产生矛盾，因此无解。

Method

- ▶ 若每个变量的两种取值都属于不同 SCC，则缩点后得到 DAG。Tarjan 的 SCC 编号按照弹栈顺序产生，越早弹出的 SCC 越接近 DAG 的汇点。
- ▶ 对每个变量，比较两个取值所属 SCC 的编号，选择拓扑意义上更靠后的可行取值，就能得到一组合法方案。
- ▶ 建图共有 $2n$ 个结点和 $2m$ 条边，总时间复杂度仍为 $\mathcal{O}(n + m)$ 。

Code

```
1 #include <stack>
2 #include <cstdio>
3 #include <cstring>
4 #include <algorithm>
5 #define MAXN 2000010
6 using namespace std;
7 int Head[MAXN], Total=0, SCC[MAXN], cnt=0, Dfn[MAXN], Low[MAXN], cnt2=0;
8 int Ans[MAXN], In1, In2, In3, In4, n, m;
9 stack <int> S;
10 struct edge{
11     int Ed, Next;
12 }Edge[MAXN<<1];
13 void Edge_Add(int St, int Ed){
14     Edge[++Total]={Ed, Head[St]};
15     Head[St]=Total;
16 }
17 void Tarjan(int Now){
18     Low[Now]=Dfn[Now]=++cnt2;
```

Code

```

1      S.push(Now);
2      for(int i=Head[Now];i;i=Edge[i].Next){
3          int To=Edge[i].Ed;
4          if(!Dfn[To]){
5              Tarjan(To);
6              Low[Now]=min(Low[Now],Low[To]);
7          }
8          else if(!SCC[To]) Low[Now]=min(Low[Now],Dfn[To]);
9      }
10     if(Low[Now]==Dfn[Now]){
11         cnt++;
12         while(S.top()!=Now){
13             SCC[S.top()]=cnt;
14             S.pop();
15         }
16         SCC[Now]=cnt;S.pop();
17     }
18 }

```

Code

```

1  int main(){
2      scanf("%d%d",&n,&m);
3      for(int i=1;i<=m;i++){
4          scanf("%d%d%d%d",&In1,&In2,&In3,&In4);
5          if(In2>In4){swap(In2,In4);swap(In1,In3);}
6          if(In2==In4&&In2==0){
7              Edge_Add(In1,In3+n);Edge_Add(In3,In1+n);
8          }
9          else if(In2==0&&In4==1){
10             Edge_Add(In1,In3);Edge_Add(In3+n,In1+n);
11         }
12         else{
13             Edge_Add(In1+n,In3);Edge_Add(In3+n,In1);
14         }
15     }
16     for(int i=1;i<=2*n;i++) if(!Dfn[i]) Tarjan(i);
17     for(int i=1;i<=n;i++){
18         if(SCC[i]==SCC[i+n]){puts("IMPOSSIBLE");return 0;}

```

Code

```
1     }
2     puts("POSSIBLE");
3     for(int i=1;i<=n;i++) Ans[i]=SCC[i]<SCC[i+n];
4     for(int i=1;i<=n;i++) printf("%d ",Ans[i]);
5     return 0;
6 }
```

Contents

连通性与 Low-Link

强连通分量

割点、桥与边双

无向图中的父边

桥

割点

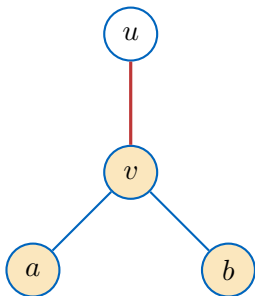
例题：删除结点后的失联点对

边双连通分量

点双连通分量

Method

- ▶ 一条无向边在链式前向星中存为两个方向。沿 (u, v) 进入 v 后，链式前向星中还会看到反向边 (v, u) ，但它不能作为绕开原边的路径。
- ▶ 令 $Total=1$ ，每次连续加入两个方向，则编号互为 $i^{\wedge}1$ 。递归时传入进入当前结点的边号，只跳过它的唯一反向边。
- ▶ 不能直接写 $To==From$ ：若 u, v 之间存在平行边，其他边仍然能够提供绕行，全部跳过会错误地把它们判成桥。



对树边 (u, v) , 若 v 的子树能够到达 u 或 u 的祖先, 就存在不经过 (u, v) 的替代路径。反之, 若整棵子树都无法回到 u , 删除这条边后子树必然与其余部分断开。

Theory

树边 (u, v) 是桥，当且仅当

$$\text{low}_v > \text{dfn}_u.$$

- ▶ 必要性：若 $\text{low}_v \leq \text{dfn}_u$ ，则 v 的子树存在一条返祖路径到达 u 或更早的祖先，从而能够绕开 (u, v) 。
- ▶ 充分性：若 $\text{low}_v > \text{dfn}_u$ ，则 v 的子树没有任何边到达 u 及其祖先；它与外部的唯一连接就是树边 (u, v) ，删除后一定不连通。

Theory

对非根结点 u ，若存在一个 DFS 儿子 v 满足

$$\text{low}_v \geq \text{dfn}_u,$$

那么 u 是割点。

- ▶ 此时 v 的子树最多只能回到 u ，无法绕过 u 到达它的祖先。删除 u 后，这棵子树会与其余部分断开。
- ▶ 若所有儿子都满足 $\text{low}_v < \text{dfn}_u$ ，每棵子树都能绕过 u 回到更早的祖先，删除 u 不会把它们分开。

Theory

- ▶ DFS 根没有祖先，所以前面的判定不能直接使用。根是割点，当且仅当它在 DFS 树中至少有两个独立儿子。
- ▶ 若只有一个儿子，删除根后其余所有结点仍在同一棵 DFS 子树中；若至少有两个儿子，不同儿子子树之间不可能存在横叉边，否则后访问的一方会在前一方的 DFS 中被搜索到。
- ▶ 对非连通图，需要从每个尚未访问的结点重新开始 Tarjan，并分别判断每棵 DFS 树的根。

Analyse

- ▶ 桥使用严格不等式，是因为返祖路径到达 u 本身已经能够绕开树边 (u, v) 。
- ▶ 割点使用非严格不等式，是因为所有绕行都必须经过 u 时，删除 u 后路径仍然失效。

$$\begin{cases} \text{low}_v > \text{dfn}_u, & (u, v) \text{ 是桥,} \\ \text{low}_v \geq \text{dfn}_u, & u \text{ 可能是割点.} \end{cases}$$

Example

P3469 [POI2008] BLO-Blockade

- ▶ 给定一张连通无向图。对于每个结点 u ，删去所有与 u 相连的边，但保留结点 u 本身。
- ▶ 求此时有多少个有序点对 (x, y) 不连通，其中 $x \neq y$ 。需要分别输出每个 u 对应的答案。

Analyse

- ▶ 固定被删除的结点 u 。每个满足 $\text{low}_v \geq \text{dfn}_u$ 的儿子子树，都会成为一个独立连通块，其大小为 $\text{Size}[\text{To}]$ 。
- ▶ 设这些子树大小之和为 S ，那么剩余部分的大小为 $n - S - 1$ 。我们逐块统计一个端点位于当前块、另一个端点位于块外的有序点对。
- ▶ 最后还要统计以 u 自身为一个端点的 $2(n - 1)$ 个有序点对。所有乘法必须使用 `long long`。

Analyse

对每个被分离的儿子子树 v ，先加入

$$\text{Size}(v)(n - \text{Size}(v)).$$

扫描结束后，令剩余部分大小为 $n - S - 1$ ，再加入

$$(n - S - 1)(S + 1) + (n - 1).$$

前一部分以分离子树为第一个端点，后一部分补上剩余区域与 u 作为第一个端点的情况，因此没有重复或遗漏。

Definition

- ▶ 在同一个边双连通分量中，任意两点之间都不存在唯一必经边。等价地，它们之间至少存在两条边不相交路径。
- ▶ 所有桥都不可能位于某个非平凡边双内部；删去全部桥后，每个剩余连通块内部不再存在桥，因此恰好是一个边双连通分量。
- ▶ 实现时先用 Tarjan 标记桥，再进行一次忽略桥的 DFS 染色即可。

Theory

- ▶ 将每个边双缩成一个结点，并保留原图中的桥。若原图连通，缩点后得到一棵树，称为桥树；一般情况下得到森林。
- ▶ 若缩点后仍有环，那么环上的任意一条桥都可以沿环绕开，与它是桥矛盾；缩点后也不会失去连通性，所以连通时必为树。
- ▶ 原图中许多“经过多少座桥”“补边消除所有桥”的问题，就转化为树上的路径或叶子问题。

Example

P2860 [USACO06JAN] Redundant Paths G

- ▶ 给定一张连通无向图。现在可以在任意两个不同结点之间加入新的无向边。
- ▶ 求至少需要加入多少条边，才能使任意两点之间都存在两条边不相交的路径。

Analyse

- ▶ 题目的要求等价于使图中不再存在桥。缩成桥树后，一次加边会让两个端点之间的整条树路径进入一个环。
- ▶ 每个叶子只有一条桥与其余部分连接，因此它至少要成为某条新增边的端点。将叶子两两配对即可同时消除两端路径上的桥。
- ▶ 若桥树有 L 个叶子，最少新增边数为 $\lceil L/2 \rceil$ 。只有一个缩点时答案为 0。

Code

```

1  #include <cstdio>
2  #include <cstring>
3  #include <algorithm>
4  #define MAXN 200010
5  #define MAXM 500010
6  using namespace std;
7  int n,m,Head[MAXN],Total=1,Dfn[MAXN],Low[MAXN],cnt=0;
8  int In1,In2;
9  bool Cut[MAXN],Bridge[MAXM<<1];
10 struct edge{
11     int St,Ed,Next;
12 }Edge[MAXM<<1];
13 void Edge_Add(int St,int Ed){
14     Edge[++Total]={St,Ed,Head[St]};
15     Head[St]=Total;
16 }
17 void Tarjan(int Now,int From){
18     Dfn[Now]=Low[Now]=++cnt;

```

Code

```

1    int Son=0;
2    for(int i=Head[Now];i;i=Edge[i].Next){
3        if(i==(From^1)) continue;
4        int To=Edge[i].Ed;
5        if(!Dfn[To]){
6            Son++;
7            Tarjan(To,i);
8            Low[Now]=min(Low[Now],Low[To]);
9            if(Low[To]>Dfn[Now]) Bridge[i]=Bridge[i^1]=true;
10           if(From&&Low[To]>=Dfn[Now]) Cut[Now]=true;
11        }
12        else Low[Now]=min(Low[Now],Dfn[To]);
13    }
14    if(!From&&Son>=2) Cut[Now]=true;
15 }
16 int main(){
17     scanf("%d%d",&n,&m);
18     for(int i=1;i<=m;i++){

```

Code

```

1      scanf("%d%d",&In1,&In2);
2      Edge_Add(In1,In2);Edge_Add(In2,In1);
3  }
4  for(int i=1;i<=n;i++) if(!Dfn[i]) Tarjan(i,0);
5  for(int i=1;i<=n;i++) printf("%d ",Cut[i]);
6  puts("");
7  for(int i=1;i<=m;i++) printf("%d ",Bridge[i<<1]);
8  return 0;
9  }
```

Code

```

1  /*
2  - Author:Ehundategh
3  - Update:2023/9/25
4  - Tile:blo.cpp
5  - You steal,I kill
6  */
7  #include <cstdio>
8  #include <cstring>
9  #include <algorithm>
10 #define MAXN 100010
11 #define MAXM 500010
12 using namespace std;
13 int Head[MAXN],Total=0;
14 int DFN[MAXN],Low[MAXN],cnt=0,Root,Size[MAXN],n,m;
15 long long Ans[MAXN];
16 bool Cut[MAXN];
17 struct edge{
18     int St,Ed;

```


Code

```

1      int Next;
2  }Edge[MAXM<<1];
3  void Edge_Add(int St,int Ed){
4      Edge[++Total]=(edge){St,Ed,Head[St]};
5      Head[St]=Total;
6  }
7  void Tarjan(int Now){
8      DFN[Now]=Low[Now]=++cnt;
9      Size[Now]=1;
10     int Son=0,Sum=0;
11     for(int i=Head[Now];i;i=Edge[i].Next){
12         int To=Edge[i].Ed;
13         if(!DFN[To]){
14             Son++;
15             Tarjan(To);
16             Size[Now]+=Size[To];
17             Low[Now]=min(Low[Now],Low[To]);
18             if(Low[To]>=DFN[Now]){

```

Code

```

1         Ans[Now] += 111 * Size[To] * (n - Size[To]);
2         Sum += Size[To];
3         if (Now != Root || Son >= 2) Cut[Now] = 1;
4     }
5 }
6 else Low[Now] = min(Low[Now], DFN[To]);
7 }
8 if (Cut[Now]) {
9     Ans[Now] += 111 * (n - Sum - 1) * (Sum + 1) + (n - 1);
10 }
11 else {
12     Ans[Now] = 111 * (n - 1) * 2;
13 }
14 }
15 int main() {
16     int In1, In2;
17     scanf("%d%d", &n, &m);
18     for (int i = 1; i <= m; i++) {

```

Code

```
1         scanf("%d%d",&In1,&In2);
2         Edge_Add(In1,In2);
3         Edge_Add(In2,In1);
4     }
5     Root=1;
6     Tarjan(1);
7     for(int i=1;i<=n;i++){
8         printf("%lld\n",Ans[i]);
9     }
10    return 0;
11 }
```

Code

```

1  #include <cstdio>
2  #include <cstring>
3  #include <algorithm>
4  #define MAXN 200010
5  #define MAXM 500010
6  using namespace std;
7  int n,m,Head[MAXN],Total=1,Dfn[MAXN],Low[MAXN],cnt=0;
8  int DCC[MAXN],cnd=0,In1,In2;
9  bool Bridge[MAXM<<1];
10 struct edge{
11     int St,Ed,Next;
12 }Edge[MAXM<<1];
13 void Edge_Add(int St,int Ed){
14     Edge[++Total]={St,Ed,Head[St]};
15     Head[St]=Total;
16 }
17 void Tarjan(int Now,int From){
18     Dfn[Now]=Low[Now]=++cnt;

```

Code

```

1   for(int i=Head[Now];i;i=Edge[i].Next){
2       if(i==(From^1)) continue;
3       int To=Edge[i].Ed;
4       if(!Dfn[To]){
5           Tarjan(To,i);
6           Low[Now]=min(Low[Now],Low[To]);
7           if(Low[To]>Dfn[Now]) Bridge[i]=Bridge[i^1]=true;
8       }
9       else Low[Now]=min(Low[Now],Dfn[To]);
10  }
11 }
12 void Paint(int Now){
13     DCC[Now]=cnd;
14     for(int i=Head[Now];i;i=Edge[i].Next){
15         int To=Edge[i].Ed;
16         if(Bridge[i]||DCC[To]) continue;
17         Paint(To);
18     }

```

Code

```

1  }
2  int main(){
3      scanf("%d%d",&n,&m);
4      for(int i=1;i<=m;i++){
5          scanf("%d%d",&In1,&In2);
6          Edge_Add(In1,In2);Edge_Add(In2,In1);
7      }
8      for(int i=1;i<=n;i++) if(!Dfn[i]) Tarjan(i,0);
9      for(int i=1;i<=n;i++) if(!DCC[i]){cnd++;Paint(i);}
10     printf("%d\n",cnd);
11     for(int i=1;i<=n;i++) printf("%d ",DCC[i]);
12     return 0;
13 }
```

Contents

连通性与 Low-Link

强连通分量

割点、桥与边双

点双连通分量
定义与边栈
例题与比较

Definition

- ▶ 一个极大的点集合是点双连通分量，当其中任意两点之间都存在两条内部点不相交路径。单独一条桥及其两个端点也视为一个点双。
- ▶ 一条边只会属于一个点双，但割点会同时属于多个点双：它正是这些部分之间唯一共同的结点。
- ▶ 因此，点双不能像边双一样只给每个结点染一个颜色，需要直接记录每个分量包含哪些结点。

Analyse

- ▶ DFS 过程中，将树边与指向祖先的返祖边压入栈中。栈中保存的是已经访问、但所属点双尚未封闭的边。
- ▶ 对树边 (u, v) ，当 $\text{low}_v < \text{dfn}_u$ 时， v 的子树仍能绕过 u 回到更早祖先，因此当前点双还要继续向上延伸。
- ▶ 当 $\text{low}_v \geq \text{dfn}_u$ 时， v 的子树无法绕过 u ，点双的边界就在 (u, v) ：从栈顶弹到这条树边即可得到一个点双。

Theory

- ▶ 被弹出的所有边都位于 v 的 DFS 子树中，并通过返祖边形成不经过内部某个单点的替代路径；它们与边界点 u 共同属于同一个点双。
- ▶ 若再向栈底多弹一条边，就会跨过 u 到达另一棵已经封闭或仍向上延伸的子树。由于 v 的子树无法绕过 u ，这两部分不能属于同一个点双。
- ▶ 因而 $\text{Low}[\text{To}] \geq \text{Dfn}[\text{Now}]$ 同时给出了割点条件与点双弹栈边界。

Method

- ▶ 弹出一条边时，将它的两个端点加入当前点双。一个结点可能在多条边中出现，所以使用当前分量编号作为时间戳去重。
- ▶ 只对满足 $\text{dfn}_v < \text{dfn}_u$ 的返祖边入栈，避免同一条无向边的两个方向重复进入边栈。
- ▶ 没有任何边的孤立点无法在弹栈过程中出现，需要单独建立一个只包含它自己的点双。

Example

P3225 [HNOI2012] 矿场搭建

- ▶ 煤矿工地由若干挖煤点和连接它们的隧道组成。需要选择一些挖煤点设置救援出口。
- ▶ 要求任意一个挖煤点坍塌后，其余挖煤点的工人仍能到达至少一个救援出口。求最少需要设置的出口数，以及达到最少出口数的方案数。

Analyse

- ▶ 对每个点双分别统计其中割点和普通点的数量。若点双不含割点，它无法通过其他部分进入，需要在内部选择至少两个不同位置；若恰含一个割点，只需要在普通点中选择一个位置。
- ▶ 这类题目的关键不是直接在原图上分类讨论，而是先把所有点双完整弹出，再根据每个分量与割点的关系独立计算贡献。
- ▶ 同一个割点会出现在多个点双中，所以不能用“每点唯一所属分量”的数组代替成员表。

Analyse

对象	核心条件	辅助结构	结果结构
SCC	$\text{low}_u = \text{dfn}_u$	点栈	DAG
桥	$\text{low}_v > \text{dfn}_u$	无	桥树
割点	$\text{low}_v \geq \text{dfn}_u$	根特判	分隔结点
边双	删除全部桥	染色	森林
点双	$\text{low}_v \geq \text{dfn}_u$	边栈	成员集合

所有算法都只进行常数 DFS，时间复杂度为 $\mathcal{O}(n + m)$ 。

Analyse

- ▶ 无向图按父结点而不是父边跳过，导致平行边被错误忽略；桥使用了非严格不等式；根割点没有单独统计 DFS 儿子。
- ▶ SCC 中用已经离栈的结点更新 low；点双中把无向边的两个方向都压入边栈；遗漏孤立点。
- ▶ 只从结点 1 开始 DFS，默认原图连通。通用模板必须遍历所有尚未访问的结点。

Code

```

1  #include <vector>
2  #include <cstdio>
3  #include <cstring>
4  #include <algorithm>
5  #define MAXN 200010
6  #define MAXM 500010
7  using namespace std;
8  int n,m,Head[MAXN],Total=1,Dfn[MAXN],Low[MAXN],cnt=0;
9  int Stack[MAXM<<1],Top=0,Tag[MAXN],cnd=0,In1,In2;
10 vector <int> DCC[MAXM];
11 struct edge{
12     int St,Ed,Next;
13 }Edge[MAXM<<1];
14 void Edge_Add(int St,int Ed){
15     Edge[++Total]={St,Ed,Head[St]};
16     Head[St]=Total;
17 }
18 void Add_Point(int Now,int x){

```


Code

```

1      if(Tag[x]==Now) return;
2      Tag[x]=Now;DCC[Now].push_back(x);
3  }
4  void Tarjan(int Now,int From){
5      Dfn[Now]=Low[Now]=++cnt;
6      for(int i=Head[Now];i;i=Edge[i].Next){
7          if(i==(From^1)) continue;
8          int To=Edge[i].Ed;
9          if(!Dfn[To]){
10             Stack[++Top]=i;
11             Tarjan(To,i);
12             Low[Now]=min(Low[Now],Low[To]);
13             if(Low[To]>=Dfn[Now]){
14                 cnd++;
15                 while(Stack[Top]!=i){
16                     Add_Point(cnd,Edge[Stack[Top]].St);
17                     Add_Point(cnd,Edge[Stack[Top]].Ed);
18                     Top--;

```

Code

```

1          }
2          Add_Point(cnd,Edge[i].St);
3          Add_Point(cnd,Edge[i].Ed);
4          Top--;
5      }
6  }
7  else if(Dfn[To]<Dfn[Now]){
8      Stack[++Top]=i;
9      Low[Now]=min(Low[Now],Dfn[To]);
10 }
11 }
12 }
13 int main(){
14     scanf("%d%d",&n,&m);
15     for(int i=1;i<=m;i++){
16         scanf("%d%d",&In1,&In2);
17         Edge_Add(In1,In2);Edge_Add(In2,In1);
18     }

```

Code

```

1   for(int i=1;i<=n;i++){
2       if(Dfn[i]) continue;
3       if(!Head[i]){cnd++;DCC[cnd].push_back(i);Tag[i]=cnd;}
4       else Tarjan(i,0);
5   }
6   printf("%d\n",cnd);
7   for(int i=1;i<=cnd;i++){
8       sort(DCC[i].begin(),DCC[i].end());
9       printf("%d ",(int)DCC[i].size());
10      for(int x:DCC[i]) printf("%d ",x);
11      puts("");
12  }
13  return 0;
14  }

```